MATA KULIAH: JARINGAN KOMPUTER DAN LAN (JARKOMLAN) • TUGAS 2

SOP & Panduan Delegasi Pengerjaan Host 2

Target Node: Host 2 (Windows • Docker Publisher & Web Client) **Kelompok:** Nicholas B., Dzaky H., Alvis S., Sayyid T.

Revisi: 1.0 (Final)

1. Ringkasan Tugas & Peran Host 2

Dokumen ini disusun sebagai panduan teknis bagi anggota tim yang memegang **Host 2**. Pada sistem terdistribusi ini, Host 2 berperan ganda sebagai **Edge IoT Node** (pengirim data) dan **Monitoring Station** (peninjau data dari Web Server di Host 1).

Layanan di Host 2	Lingkungan Eksekusi	Protokol & Port	Tujuan Komunikasi
MQTT Publisher	Docker Container	MQTT over TCP / Port `1883`	Kirim telemetri berkala ke Broker Mosquitto (Host 1)
Web Client	Browser & Python CLI	HTTP / Port `8080`	Akses dashboard data pada Web Server Linux VM (Host 1)

> **Prinsip Komunikasi Lintas Perangkat:**
> Kedua laptop (Host 1 dan Host 2) harus berada dalam satu jaringan lokal (Wi-Fi, Switch, atau Hotspot yang sama). Host 2 tidak menjalankan database atau broker; seluruh penyimpanan dan pemrosesan terpusat di Host 1.

2. Normalisasi & Standarisasi Kontrak Data (Anti-Konflik)

Untuk menghindari konflik kegagalan pembacaan (seperti *parsing error*, *type mismatch*, nilai `NULL`, atau data duplikat), ditetapkan standarisasi kontrak pertukaran data yang mengikat antara Publisher (Host 2) dan Ingestion Subscriber (Host 1).

A. Normalisasi Skema JSON Payload (Data In-Flight)

Setiap pesan yang dikirim oleh Publisher **wajib berupa string JSON valid** (UTF-8) dengan struktur kunci yang sudah dinormalisasi berikut:

Key Field	Tipe Data	Status	Format / Satuan	Deskripsi & Aturan Validasi
`device_id`	String	**WAJIB**	Alfanumerik (Huruf/Angka/Strip)	Identitas unik node pengirim (misal: `"HOST2-IOT-NODE"`). Maksimal 64 karakter.
`seq`	Integer	**WAJIB**	Bilangan bulat ≥ 1	Nomor urut paket pengiriman untuk mendeteksi *packet drop* atau *re-ordering*.
`temperature`	Float	**WAJIB**	Derajat Celcius (°C)	Nilai desimal numerik (presisi 2 angka di belakang koma, misal `28.75`). Rentang: `20.00 - 45.00`.
`humidity`	Float	**WAJIB**	Persentase Relatif (%)	Nilai desimal numerik (misal `65.40`). Rentang: `30.00 - 95.00`.
`voltage`	Float	**OPSIONAL**	Volt (V)	Tegangan baterai/daya sensor node (misal `3.30`). Rentang: `3.00 - 3.50`.
`source`	String	**OPSIONAL**	Teks Bebas	Keterangan asal lingkungan (misal: `"Host 2 Windows Container"`).

> **Perhatian Normalisasi Nilai:**
> Jangan mengirim angka dalam format string (contoh yang **salah**: `"temperature": "28.5"`). Kirimkan langsung sebagai tipe numerik (contoh yang **benar**: `"temperature": 28.5`). Jangan mengganti nama kunci menjadi bahasa lain (misal `suhu` atau `temp`) karena Subscriber hanya mengenali key `temperature`.

Contoh Payload JSON Baku:

```
{
  "device_id": "HOST2-IOT-NODE",
  "seq": 14,
  "temperature": 29.40,
  "humidity": 68.10,
  "voltage": 3.32,
  "source": "Host 2 Windows Docker Container"
}
```

B. Sinkronisasi dengan Normalisasi Database MySQL di Host 1

Di sisi Host 1, data yang masuk dari Host 2 otomatis dipetakan ke dalam tabel MySQL `sensor_telemetry`. Struktur ini telah memenuhi kaidah **3rd Normal Form (3NF)**:

- **1NF (First Normal Form):** Setiap kolom bernilai atomik (atribut suhu, kelembapan, dan voltase berdiri sendiri, bukan array/composite). Primary key unik berupa `id INT AUTO_INCREMENT`.
- **2NF (Second Normal Form):** Seluruh kolom non-kunci bergantung secara fungsional penuh pada Primary Key transaksi.
- **3NF (Third Normal Form):** Tidak terdapat ketergantungan transitif antar-atribut non-kunci.

C. Standarisasi Topik & Parameter MQTT

Parameter	Nilai Standar	Keterangan
Topic Target	`jarkom/sensor/telemetry`	Subscriber Host 1 mendengarkan pola wildcard: `jarkom/sensor/#`
QoS (Quality of Service)	`0` (At most once) atau `1` (At least once)	Gunakan QoS `0` untuk throughput cepat atau QoS `1` untuk jaminan sampai.
Retain Flag	`False`	Pesan tidak disimpan permanen di broker sebagai retain message.
Interval Pengiriman	Setiap `3 detik` (Default)	Dapat diatur lewat environment variable `PUBLISH_INTERVAL`.

3. Langkah Pengerjaan di Host 2 (Hands-on Step-by-Step)

Langkah 1: Persiapan File Proyek

Pastikan Anda telah menyalin folder `host2/` dari repositori kelompok ke laptop Host 2. Struktur berkas yang Anda perlukan:

```
host2/
├── docker/
│   ├── docker-compose.yml          # Konfigurasi container
│   └── publisher/
│       ├── Dockerfile              # Setup image Python
│       ├── requirements.txt        # Dependensi paho-mqtt
│       └── publisher.py            # Logika pengiriman data periodik
└── web_client/
    └── test_web_client.py          # Script uji coba client via terminal
```

Langkah 2: Sinkronisasi Alamat IP Host 1

1. Tanyakan kepada pemegang **Host 1** alamat IP LAN miliknya (hasil `ipconfig` di Host 1, misal: `192.168.1.10`).
2. Buka berkas `host2/docker/docker-compose.yml` menggunakan teks editor (VS Code / Notepad).
3. Ubah nilai `MQTT_BROKER_HOST` sesuai IP Host 1:

```
services:
  mqtt-publisher:
    build:
      context: ./publisher
      dockerfile: Dockerfile
    container_name: jarkom-mqtt-publisher-host2
    environment:
      MQTT_BROKER_HOST: "192.168.1.10"   # <-- GANTI DENGAN IP WINDOWS HOST 1
      MQTT_BROKER_PORT: "1883"
      MQTT_TOPIC: "jarkom/sensor/telemetry"
      DEVICE_ID: "HOST2-IOT-NODE"
      PUBLISH_INTERVAL: "3"
    restart: unless-stopped
```

Langkah 3: Jalankan Container Publisher di Docker

Buka PowerShell atau CMD di Host 2, masuk ke direktori folder `host2/docker` dan jalankan:

```
cd "path\ke\folder\host2\docker"
docker compose up -d --build
```

Langkah 4: Pantau Log Pengiriman Data

Pastikan container berhasil terhubung dan mengirim paket data setiap 3 detik dengan melihat log:

```
docker logs -f jarkom-mqtt-publisher-host2
```

Ciri Berhasil: Muncul pesan `Berhasil terhubung ke Broker di 192.168.1.10:1883` dan baris `[1] Data terkirim ke jarkom/sensor/telemetry: {...}`.

Langkah 5: Eksekusi Web Client (Uji Penerimaan di Host 2)

Ada 2 cara memvalidasi peran Web Client dari Host 2:

- **Opsi A (Via Browser Google Chrome / Edge):**
  Buka browser di Host 2 dan ketik alamat Web Server Host 1: `http://<IP_VM_LINUX_HOST1>:8080` (misal: `http://192.168.1.15:8080`).
  Verifikasi: Dashboard web akan menampilkan tabel telemetri dengan Device ID `HOST2-IOT-NODE` yang bertambah secara real-time.
- **Opsi B (Via Python CLI Web Client):**
  Jalankan script verifikasi di terminal Host 2:

  ```
  python host2\web_client\test_web_client.py
  ```

  Catatan: Script akan mencetak respon HTTP 200 OK beserta tabel ringkasan data yang tersimpan di server.

4. Checklist Verifikasi & Troubleshooting Jaringan

Gejala Kendala	Penyebab Utama	Langkah Solusi Cepat
Koneksi Timeout / Refused ke Port 1883	Windows Firewall di Host 1 memblokir paket masuk, atau IP Broker salah.	1. Uji konektivitas di PowerShell Host 2: `Test-NetConnection -ComputerName 192.168.1.10 -Port 1883` 2. Pastikan Host 1 sudah menjalankan `setup_firewall.ps1`.
Tidak Bisa Ping antar Host	Fitur *AP Isolation* aktif pada Wi-Fi publik/kampus, memblokir lalu lintas antar-klien.	Gunakan **Hotspot / Tethering dari HP** untuk menghubungkan kedua laptop ke satu jaringan lokal bersama tanpa restriksi isolasi.
Web Server tidak bisa dibuka dari Host 2	Adapter jaringan VM di Host 1 masih dalam mode NAT, bukan Bridged.	Minta rekan Host 1 mengubah pengaturan adapter VM di VirtualBox menjadi **Bridged Adapter** agar VM mendapat IP LAN sendiri.
Data tidak muncul di Web Server	Subscriber di Host 1 belum membaca pesan, atau format JSON tidak sesuai.	Periksa log subscriber di Host 1: `docker logs -f jarkom-mqtt-subscriber`. Pastikan key JSON sesuai standar di Bab 2.

Pengambilan Bukti Laporan Praktikum (Evidence Checklist):

Sebelum mematikan layanan, anggota pemegang Host 2 wajib mengambil tangkapan layar (screenshot):

1. Tangkapan layar terminal perintah `docker logs -f jarkom-mqtt-publisher-host2` yang menunjukkan paket data sukses terkirim.
2. Tangkapan layar browser di Host 2 yang menampilkan antarmuka dashboard Web Server Host 1 berisi data dari Host 2.
3. *(Nilai Tambah Jarkom)* Tangkapan layar **Wireshark** di Host 2 dengan filter `mqtt || http` yang memperlihatkan paket TCP MQTT Publish (port 1883) dan HTTP GET (port 8080).
